

BÁO CÁO REVIEW CODE: MODULE AUTH & USER

1. Single Responsibility Principle (SRP) - Nguyên lý đơn nhiệm

Đây vẫn là nguyên lý bị vi phạm nhiều nhất trong thiết kế hiện tại.

- Vi phạm: UserService.resolveRoles
 - Vấn đề: UserService vẫn tự ý tạo Role mới nếu chưa tồn tại.
 - Đoạn code: `roleRepository.findByName(roleName).orElseGet(() -> roleRepository.save(...))`
 - Tác động: UserService đang ôm đồm cả việc quản lý vòng đời của Role. Nếu Role có thêm thuộc tính (ví dụ description, permissions), UserService sẽ phải sửa đổi theo.
 - Khắc phục: User chỉ nên được gán những Role đã tồn tại. Nếu Role không tồn tại, hệ thống nên báo lỗi. Việc tạo Role thuộc về RoleService.
- Vi phạm: AuthController xử lý Cookie logic
 - Vấn đề: Controller đang chứa quá nhiều logic chi tiết về cấu hình Cookie (`setHttpOnly`, `setSecure`, `setMaxAge`, `SameSite`).
 - Tác động: Nếu sau này cần thay đổi cơ chế lưu token (ví dụ chuyển sang Header, hoặc LocalStorage phía client), bạn phải sửa Controller.
 - Khắc phục: Đẩy logic tạo Cookie vào một CookieUtil hoặc TokenCookieService.

2. Open/Closed Principle (OCP) - Nguyên lý Đóng/Mở

Code chưa sẵn sàng cho việc mở rộng luồng xác thực mà không phải sửa đổi code cũ.

- Vi phạm: AuthService.login
 - Vấn đề: Logic xác thực bị đóng cứng với `UsernamePasswordAuthenticationToken`.
 - Đoạn code:

```
authenticationManager.authenticate(  
    new UsernamePasswordAuthenticationToken(request.getEmail(),  
    request.getPassword());
```

- Tương lai: Yêu cầu bổ sung có đề cập đến "Thẻ nội địa" cho thanh toán, điều này gợi ý hệ thống sẽ mở rộng. Tương tự, nếu cần thêm đăng nhập qua Google/Facebook hoặc LDAP, bạn sẽ phải sửa thẳng vào hàm login này.
- Khắc phục: Sử dụng Strategy Pattern. Inject một list các AuthStrategy (`BasicAuth`, `OAuth2`, `OTP...`), hàm login chỉ việc gọi strategy phù hợp.
- Vi phạm: SecurityConfig

- Vấn đề: Các đường dẫn và Role được hardcode trực tiếp (`.requestMatchers("/api/admin/**").hasRole("ADMIN")`).
- Tác động: Khi thêm Role mới (ví dụ "SALES_MANAGER") hoặc thay đổi đường dẫn API, phải mở file config này ra sửa, dễ gây lỗi regression (làm hỏng các config cũ).
- Khắc phục: Externalize cấu hình này ra file .yml hoặc Database và load động vào lúc runtime.

3. Dependency Inversion Principle (DIP) - Nguyên lý đảo ngược phụ thuộc

Module cấp cao đang phụ thuộc vào chi tiết cài đặt của module cấp thấp.

- Vi phạm: AuthService ép kiểu UserPrincipal
 - Vấn đề: AuthService phụ thuộc trực tiếp vào class cụ thể UserPrincipal.
 - Đoạn code: **UserPrincipal principal = (UserPrincipal) auth.getPrincipal();**
 - Tác động: Spring Security trả về interface UserDetails hoặc Object. Việc ép kiểu cứng (casting) này khiến AuthService không thể hoạt động nếu Authentication Provider trả về một loại Principal khác (ví dụ OidcUser của Google Login).
 - Khắc phục: Tương tác thông qua interface UserDetails hoặc tạo một Adapter để lấy thông tin User ID.

4. Interface Segregation Principle (ISP) - Nguyên lý phân tách interface

- Vấn đề: UserService
 - Vấn đề: Class này phục vụ quá nhiều loại client khác nhau:
 - Client là Admin: cần listUsers, lockUser, unlockUser.
 - Client là User thường: cần updateUser, changePassword.
 - Client là Auth module: cần createUser (register).
 - Tác động: Một Controller dành cho User profile (ProfileController) khi inject UserService sẽ nhìn thấy cả các hàm nguy hiểm như lockUser.
 - Khắc phục: Tách thành các interface nhỏ: IUserAdminService, IUserProfileService, IUserRegistrationService.

TỔNG KẾT & HƯỚNG SỬA

Dựa trên mã nguồn mới nhất, tôi đề xuất lộ trình refactor để đạt chuẩn SOLID như sau:

1. Fix SRP:
 - Sửa UserService.resolveRoles: Chỉ findByName, nếu không thấy thì throw Exception.
2. Bước 2 (Fix DIP & OCP):
 - Trong AuthService: Thay vì ép kiểu (UserPrincipal), hãy viết một phương thức tiện ích SecurityUtils.getCurrentUserId() để ẩn giấu việc tương tác với SecurityContext và UserPrincipal.
 - Xem xét áp dụng Strategy Pattern cho hàm login nếu dự án có kế hoạch hỗ trợ Social Login sớm.
3. Bước 3 (Clean Code):
 - Đưa logic tạo Cookie trong AuthController ra một utility class.

- SecurityConfig: Gom nhóm các URL rules lại cho gọn gàng hơn hoặc tách ra config riêng.

Code hiện tại đã hoạt động (Functional), nhưng về mặt thiết kế (Design) thì độ kết dính (Coupling) vẫn cao và tính đơn nhiệm (Cohesion) chưa đạt, đặc biệt là sự lẫn lộn giữa Authentication logic và Data Management logic.